

WiX Toolkit walkthrough and Publishing app to Microsoft Store

1. Introduction

This document goes through the steps to install and use the WiX Toolkit to create Windows Installer packages. The WiX Toolkit is a toolkit that allows developers to create Windows Installer packages from XML source code. It is used with Visual Studio add-ons to facilitate the creation and management of installer projects.

In second part, the document explains how to publish an application to Windows Store using Windows App Packaging project.

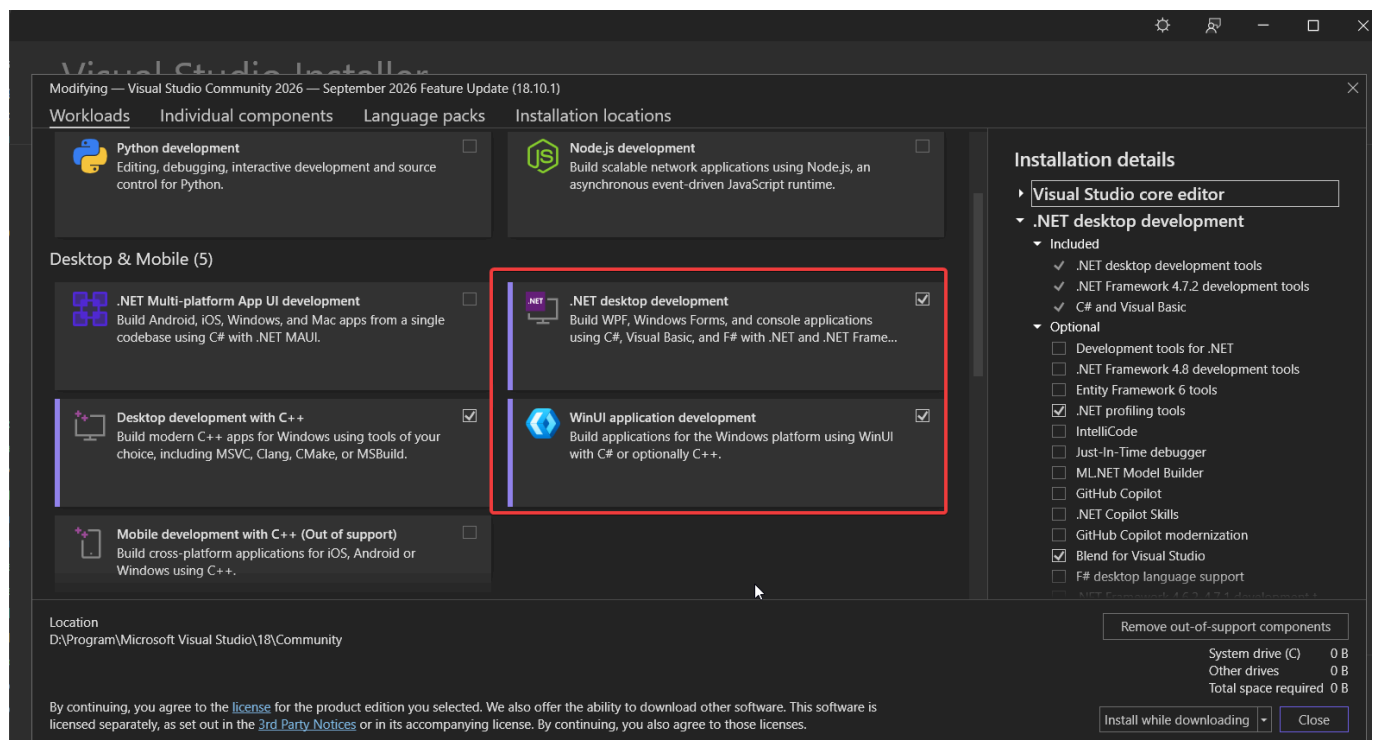
2. Install Visual Studio

You need to have the following installed on your machine:

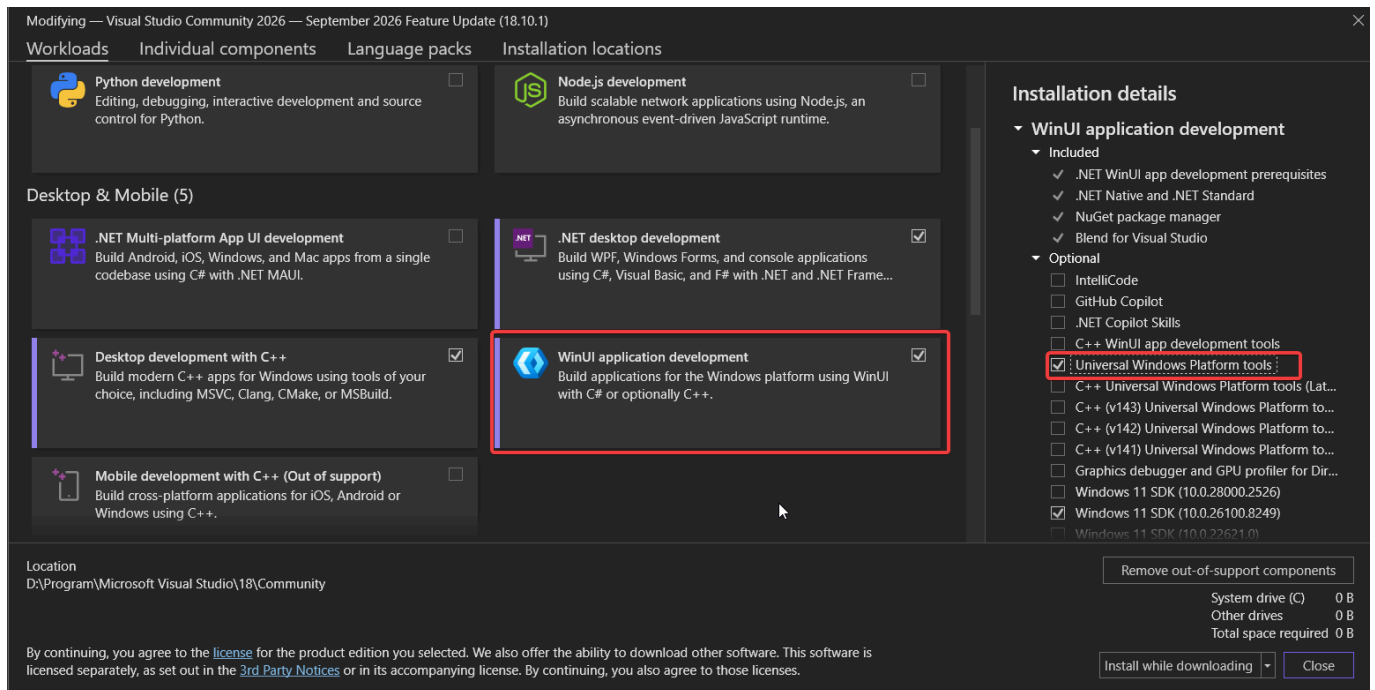
- .NET SDK installed on your machine
- Visual Studio with the necessary workloads for desktop development

To install Visual Studio go to the official [Visual Studio website](#)

Select .NET Desktop Development workload (for .NET SDK and application development) and WinUI app development workload (for packaging and deploying Windows apps to Microsoft Store)



In WinUI app development workload, make sure to check the box to install it.



3. WiX Toolkit installation

Using the .NET CLI, you can install the WiX Toolkit by running the following commands:

```
dotnet nuget add source https://api.nuget.org/v3/index.json --name nuget.org
dotnet nuget list source
dotnet tool install --global wix
```

```
C:\Users\Dung Huynh>dotnet nuget add source https://api.nuget.org/v3/index.json --name nuget.org
error: The name specified has already been added to the list of available package sources. Provide a unique name.

C:\Users\Dung Huynh>dotnet nuget list source
Registered Sources:
1. nuget.org [Enabled]
   https://api.nuget.org/v3/index.json
2. Microsoft Visual Studio Offline Packages [Enabled]
   C:\Program Files (x86)\Microsoft SDKs\NuGetPackages\

C:\Users\Dung Huynh>dotnet tool install --global wix
You can invoke the tool using the following command: wix
Tool 'wix' (version '7.0.0') was successfully installed.
```

Go to the Visual Studio Add-ons Marketplace to download and install the FireGiant HeatWave extension:

[FireGiant HeatWave extension](#)



HeatWave for Visual Studio

FireGiant | 161,522 installs | ★★★★★ (15) | Free

HeatWave is the Visual Studio extension for the WiX Toolset.

[Download](#)

[Overview](#) | [Rating & Review](#)

[HeatWave](#) is FireGiant's Visual Studio extension for the WiX Toolset. HeatWave Community Edition is available free of charge and provides:

- Conversion of WiX v3 projects and authoring
- Building of modern WiX SDK-style projects
- Project and item templates
- Property pages to control how the project builds

HeatWave peacefully coexists with the WiX v3 Visual Studio extension, which allows you to mix and match old and new versions of WiX projects in a solution. That way, you can gradually convert old WiX v3 projects to the latest version of WiX.

Categories

[Tools](#) [Build](#) [Coding](#) [Setup & Deployment](#)

Tags

[MSI](#) [Setup](#) [Windows Installer](#)
[Windows Installer XML](#) [WiX](#)

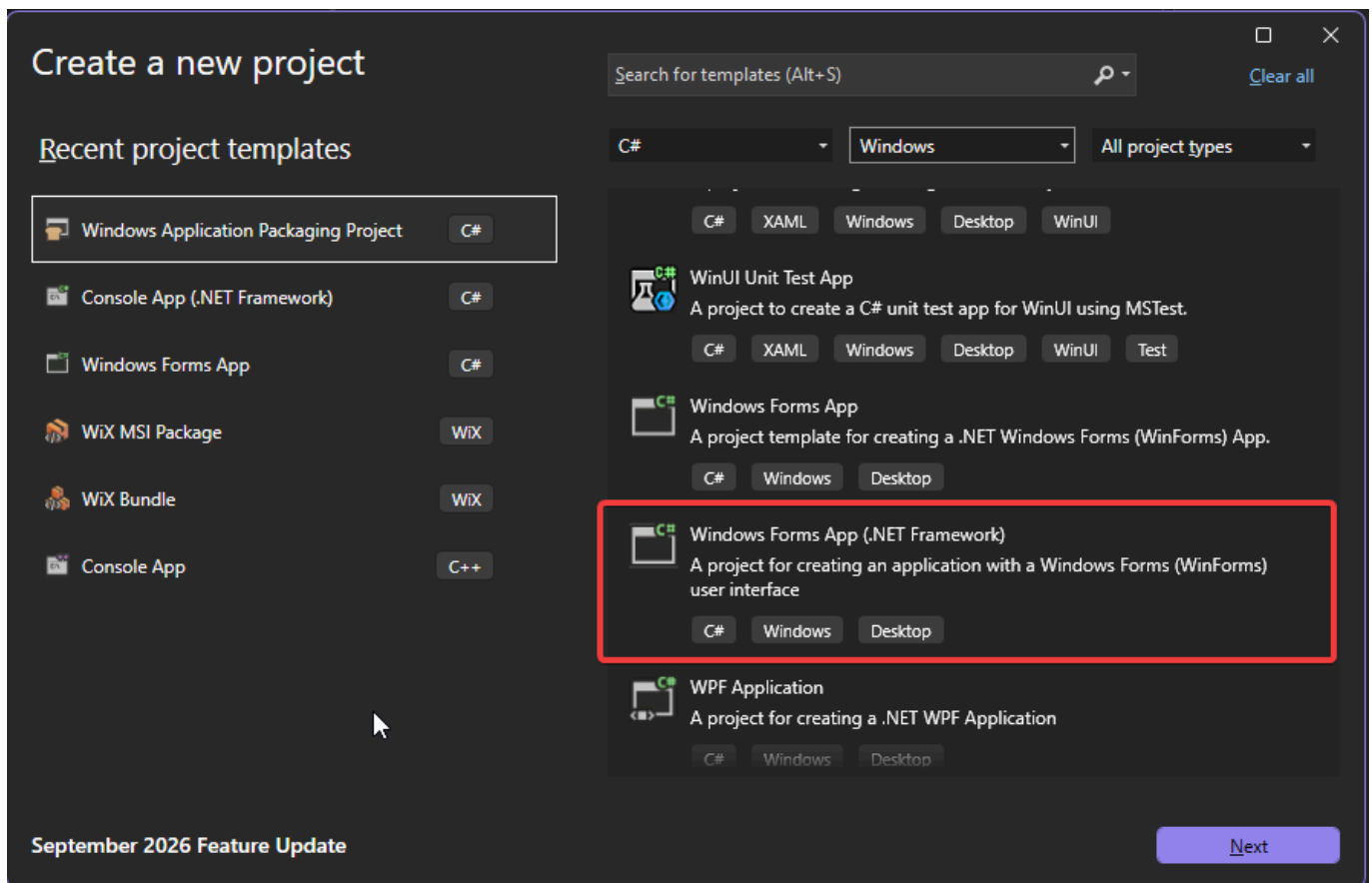
Works with

Visual Studio 2026 (amd64, arm64), 2022 (amd64, arm64)

Resources

Create a sample project

1. Open Visual Studio and create a new project with "File" --> "New" --> "Project" and in the new window click "Visual C#" then "Windows Desktop" on the sidebar. And select Windows Forms App (.NET Framework). Down the bottom ensure the framework is .NET Framework 4.6.2 or newer. Finally click "Ok" to create the new solution.

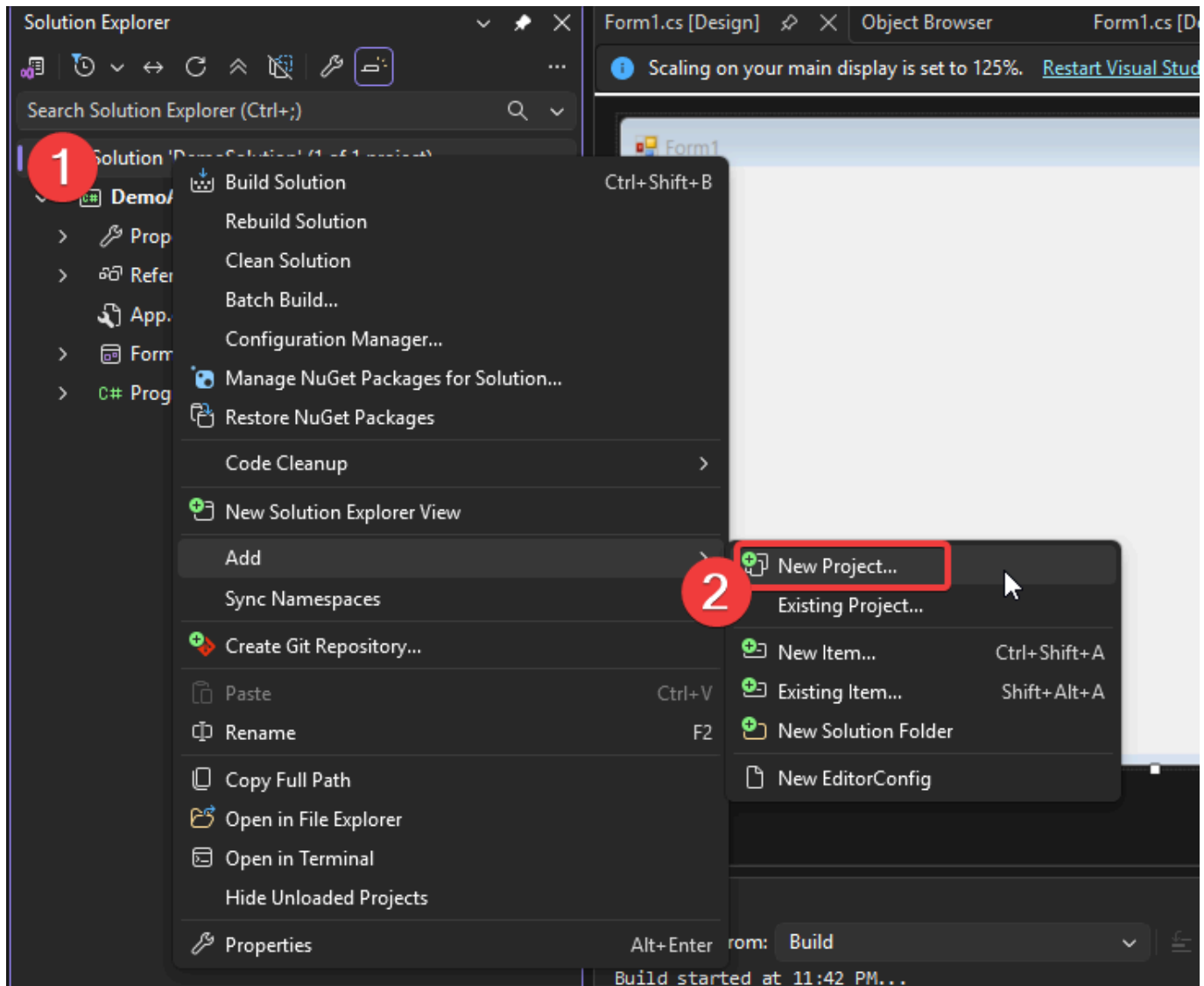


2. Test the project template works by clicking start. Wait a bit and if a window pops up that looks like the form1.cs then it has built successfully.

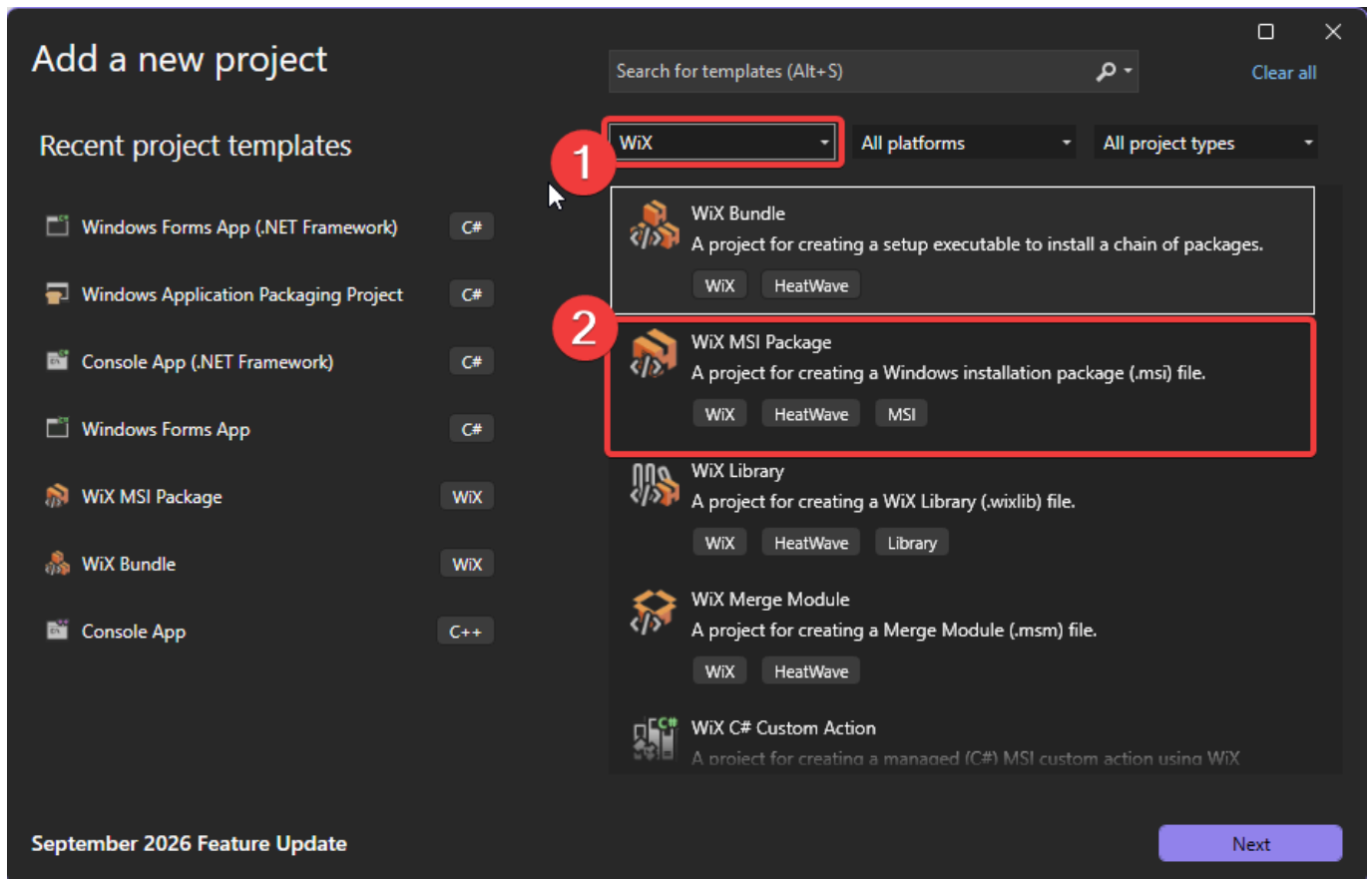
If you're familiar with C# and Windows Forms, then feel free to customize the window. But don't spend too much time on it!

5. Steps to Use the WiX Toolkit to Create Windows Installer Packages

1. Right click on the Solution in the Solution Explorer and go "Add" --> "New Project"

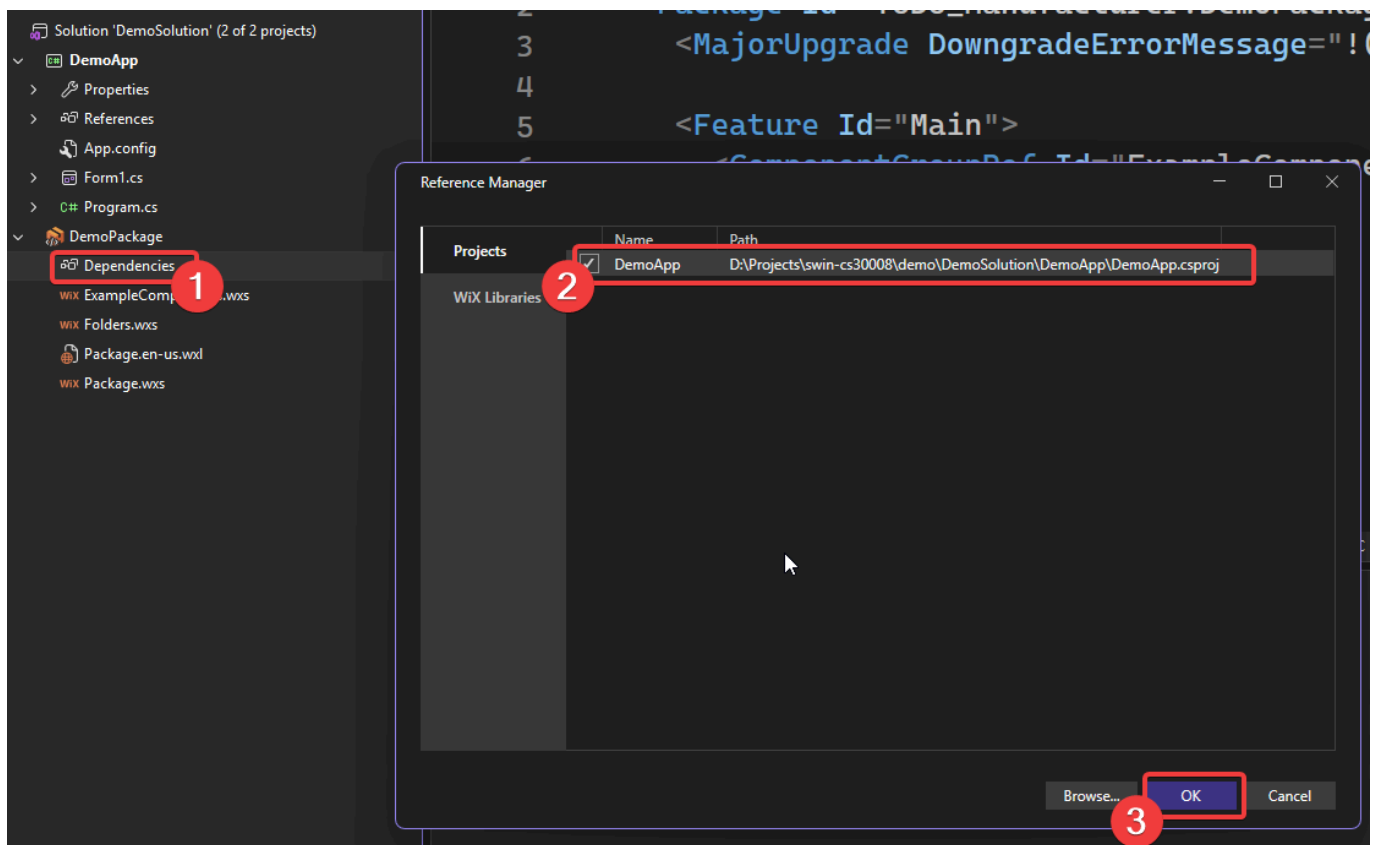


2. In the new window click "WiX". Then select "WiX MSI Package" Name the installer project and click "OK"



This will add a WiX project to the solution.

3. Create a reference to the application we want to package in the WiX project. Do this by right clicking references and selecting add reference.



4. In the newly created ExampleComponents.wxs file, find the tag. Inside that tag, we need to update a tag for everything that must be included in the installer. For our simple application, we only need to create one for the executable file. By default, the executable file generated by Visual Studio is named after the project. E.g., if the project is ExampleDesktop, the executable that is built is called ExampleDesktop.exe. The format for the tag is:

```
<Component>
  <File Source="c:/Path/To/Filename.exe"/>
</Component>
```

We can replace the Filename.exe with ExampleDesktop.exe:

```
<Component>
  <File Source="c:/Path/To/Filename.exe"/>
</Component>
```

The Source property describes where the file to be included is. Because this can change based on the projects settings we can future proof it by using the preprocessor to generate it for us.

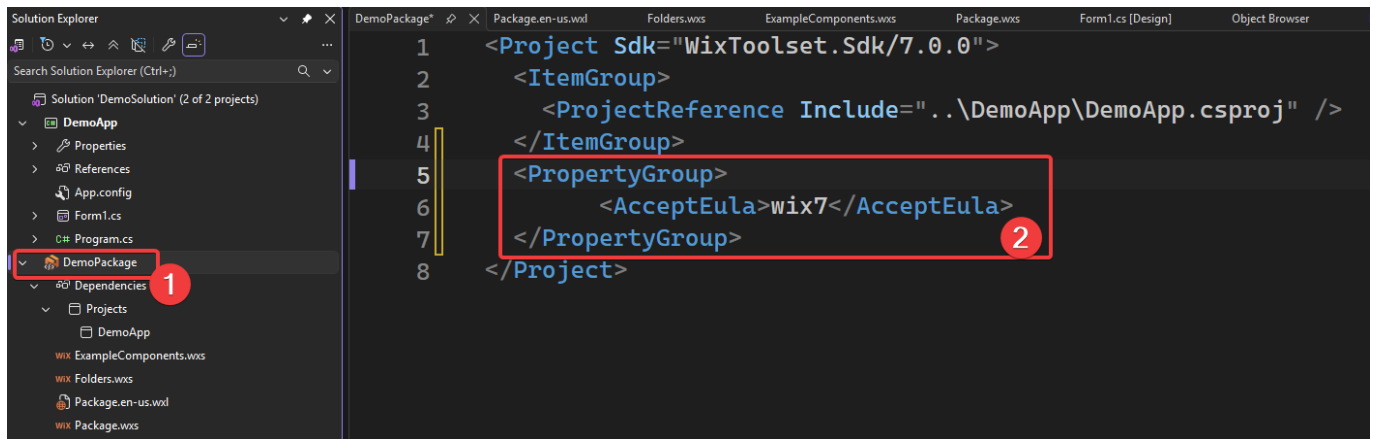
```
Source="$$(var.DemoApp.TargetDir)\DemoApp.exe"
```

When the compiler runs it will evaluate var.Demo.TargetDir to the target directory of the DemoApp project.

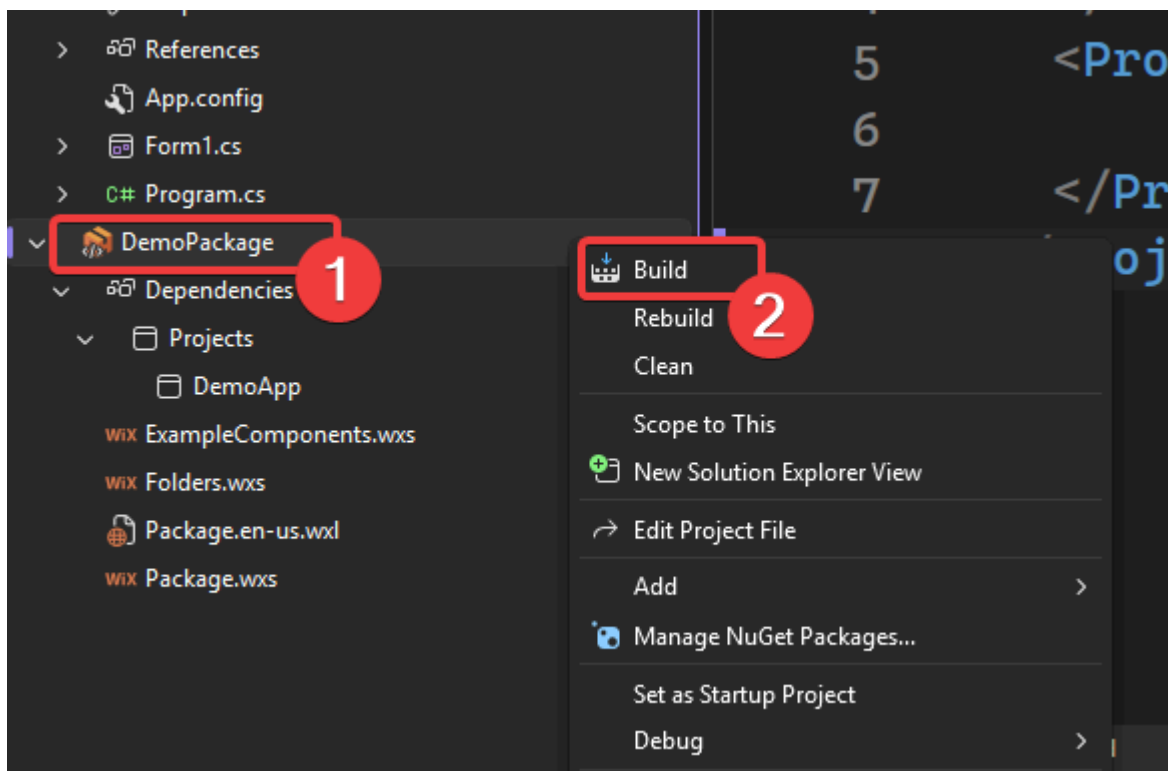
5. Final touches in the Package.wxs file.

- In the tag,
 - Put your name as the manufacturer e.g. Manufacturer="ICT Student"
 - Put the package name as appropriate e.g. Name="Demo Application"
 - Set the name to something appropriate. This is what will be displayed in Windows "add remove programs" settings.
- Find the tag with the Id="INSTALLFOLDER". Set the name to something appropriate, this will be the name of the directory created in Program files. e.g. if Name="My Example Installer" it will install the program to "c:/Program Files/My Example Installer"
- You also add the Accept License Agreement (EULA) by adding the following lines in the project; otherwise, you will get an error: **You must accept the Open Source Maintenance Fee (OSMF) EULA to use WiX Toolset v7. For instructions, see <https://wixtoolset.org/osmf/>**

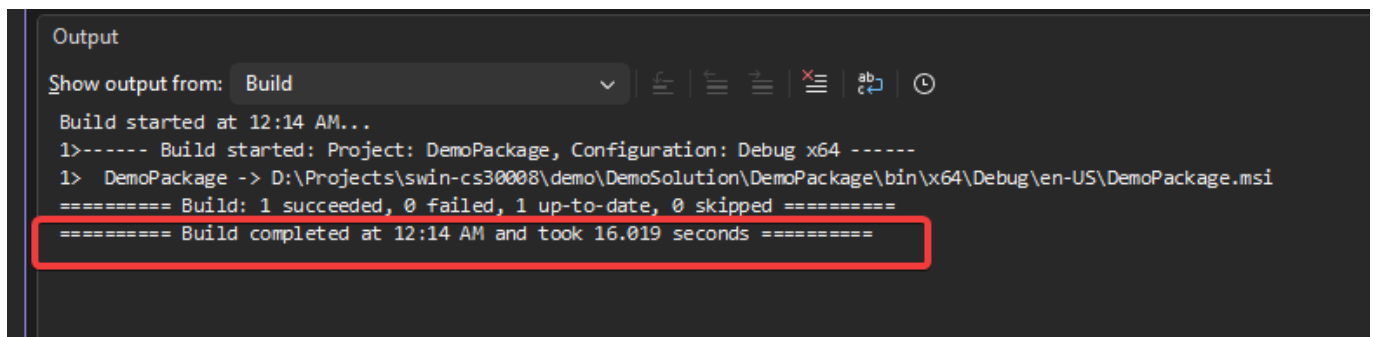
```
<PropertyGroup>
  <AcceptEula>wix7</AcceptEula>
</PropertyGroup>
```



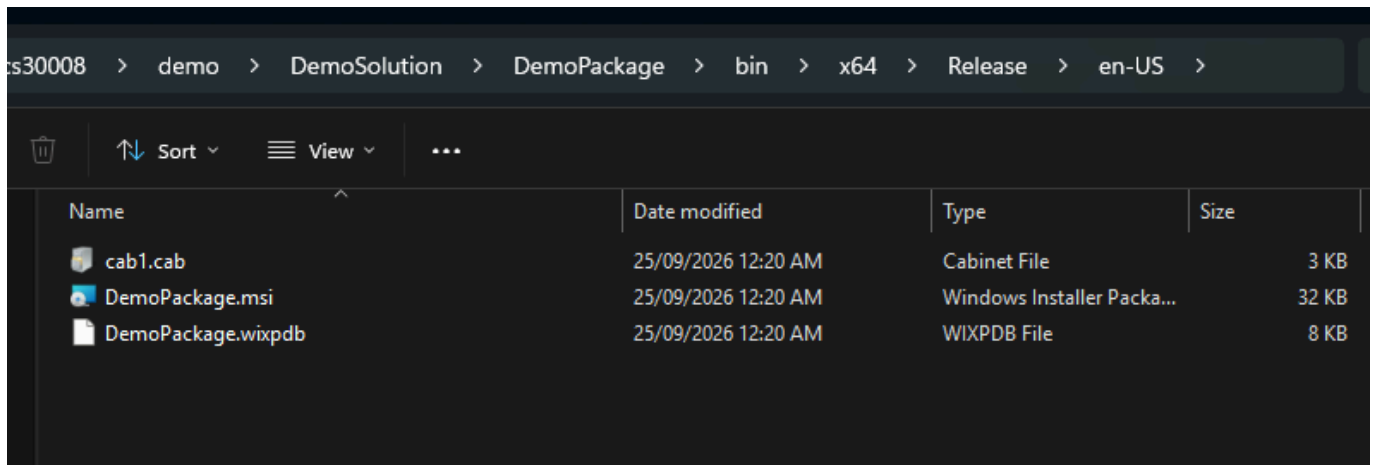
6. Ready to build. Right click on the DemoPackage project in the solution explorer and click "Build"



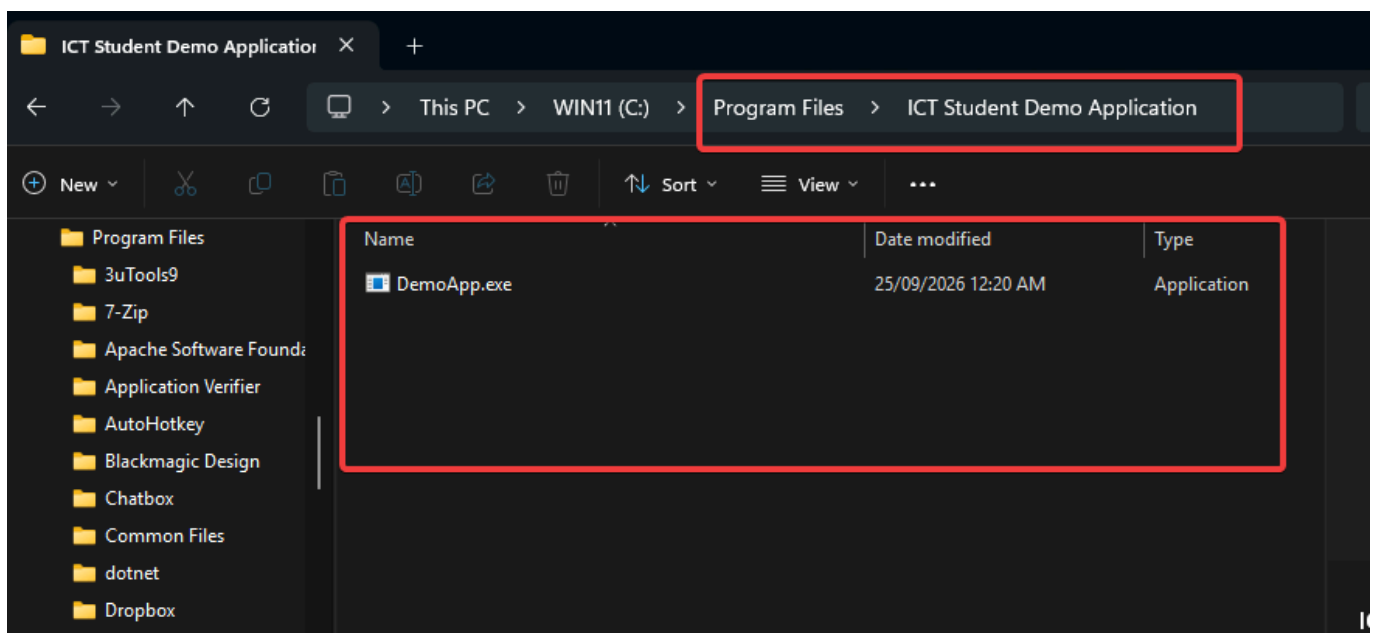
If all went well it should have built successfully.



7. Test your installer. Navigate to the directory of the Visual Studio Solution. You can do this by right clicking at the top of the solution explorer and selecting "Open in File Explorer" Navigate to the WiX project (in the example it's DemoPackage) then DemoPackage/bin/x64/release/en-US and you'll find 3 files including the .msi file which is the installer.



Double click on the installer to install your application. Check that the installation completes successfully and that the application runs as expected.

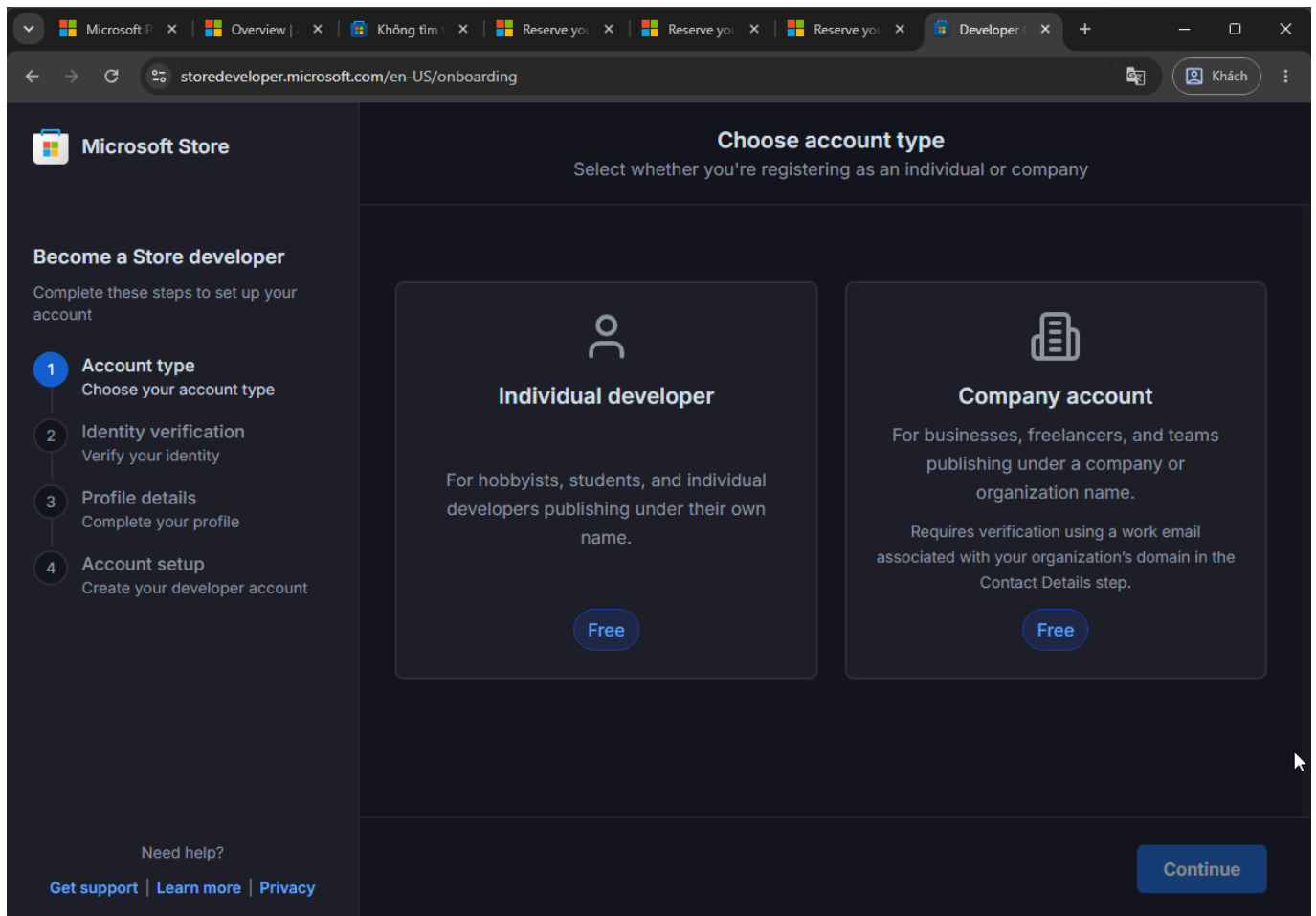


Reference: <https://docs.firegiant.com/wix/using-wix/>

6. Universal Windows App Packaging - Publish app to Microsoft Store

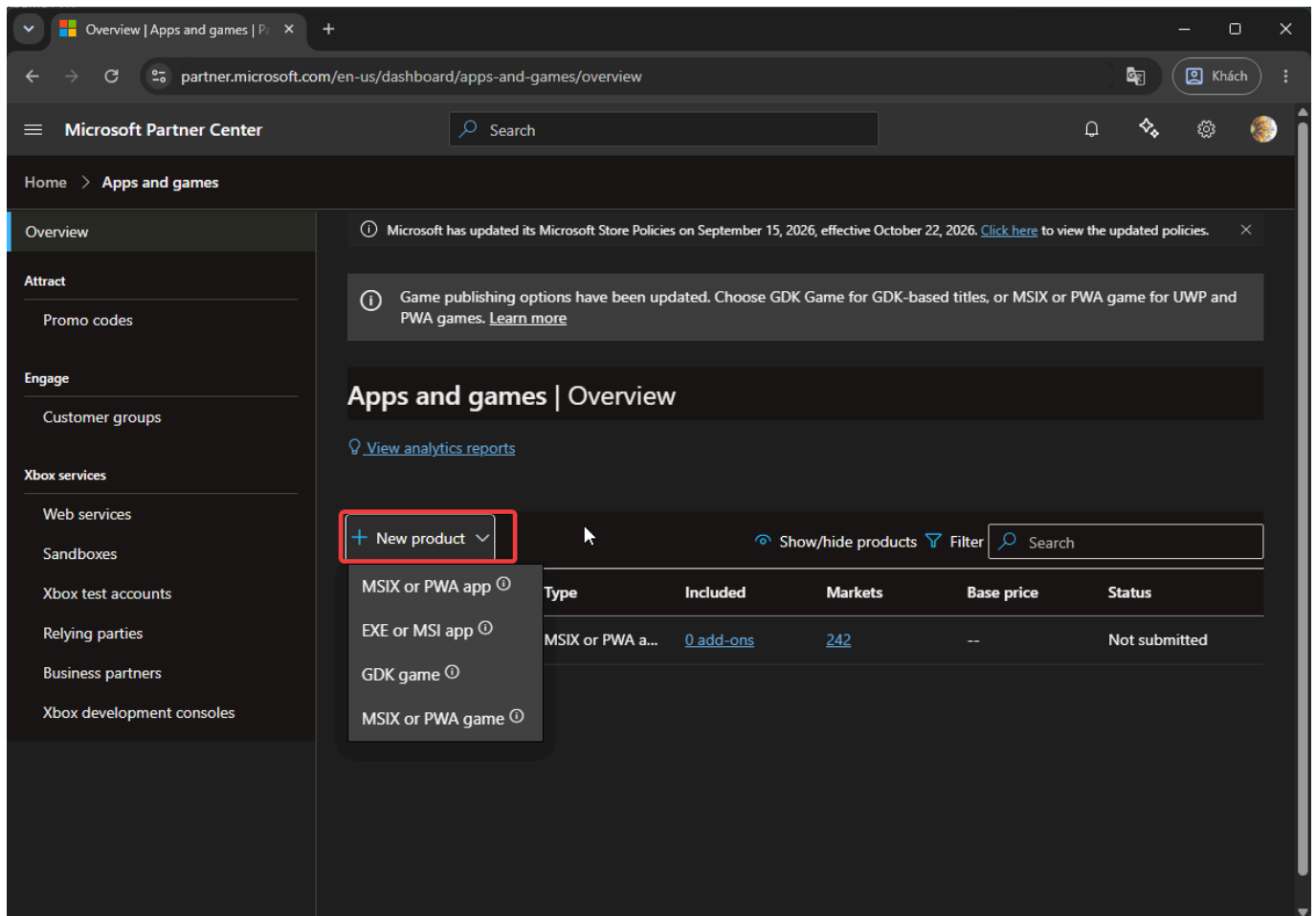
In addition to traditional installers Microsoft also now has the Windows store. These installers are quite different and far more restrictive than traditional installers. The process includes creating a developer account, reserving an app name, packaging the app, and submitting it to the Microsoft Store.

1. Create Store Developer Account. Go to the [Microsoft Store Developer Center](#) and sign up for a developer account if you don't already have one. Note: you should register with individual developer account type and you need to submit personal identification for verification.



After registering successfully, you can proceed to create and submit your application packages to the Microsoft Store.

Go to [Microsoft Partner Center](#), then choose Apps and games. Select **New product** --> Choose MSIX or PWA app to create an app submission.



Reserve your app name in the Microsoft Store before submitting your application package. This ensures that your desired app name is available and reserved for your use.

Create your app by reserving a name

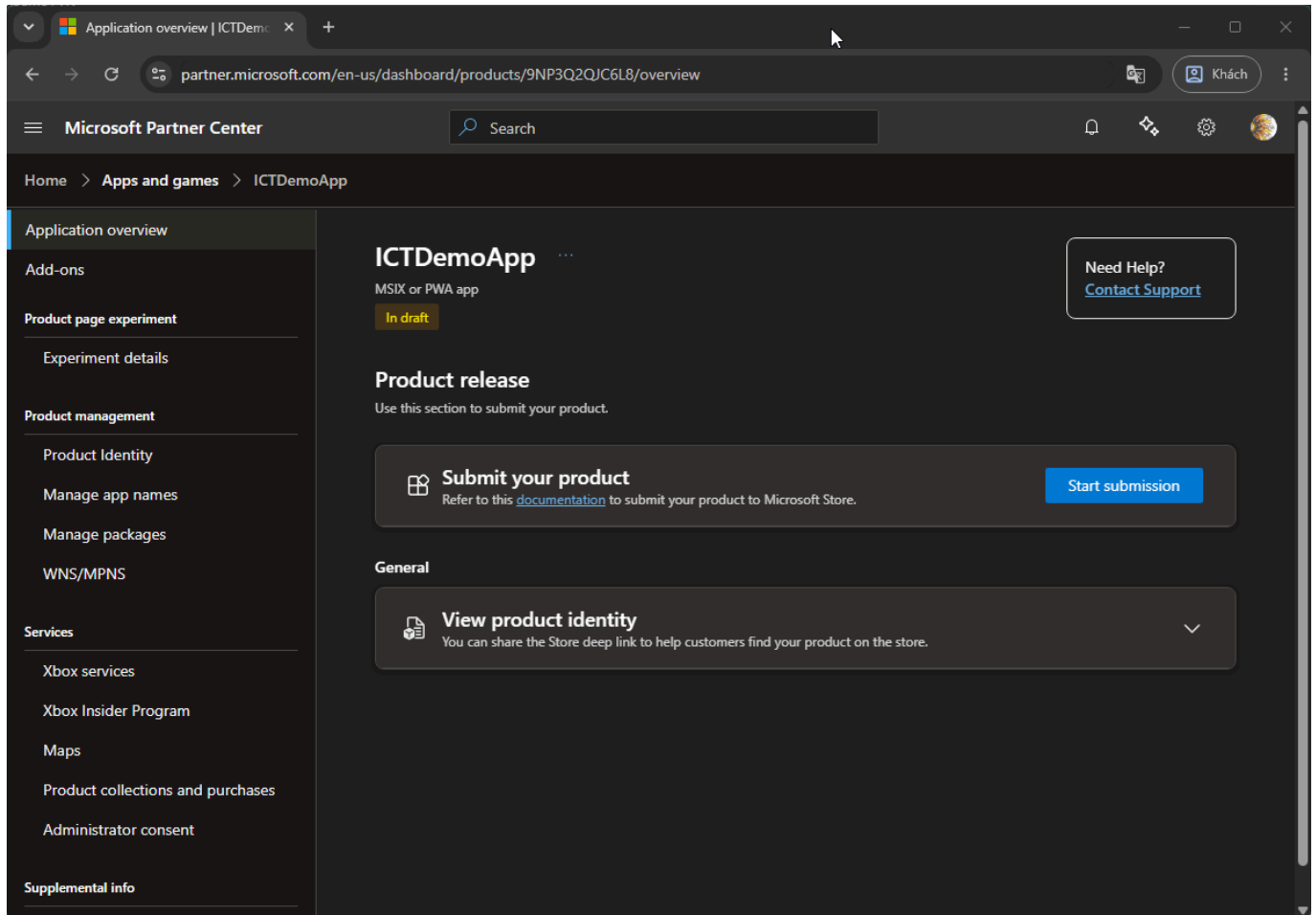
Once you reserve a name, your app will be provisioned for services like push notifications and you can start defining add-ons.

Make sure you have the rights to use any name you reserve. You must submit this app to the Microsoft Store within three months, or you'll lose your name reservation. [Learn more](#)

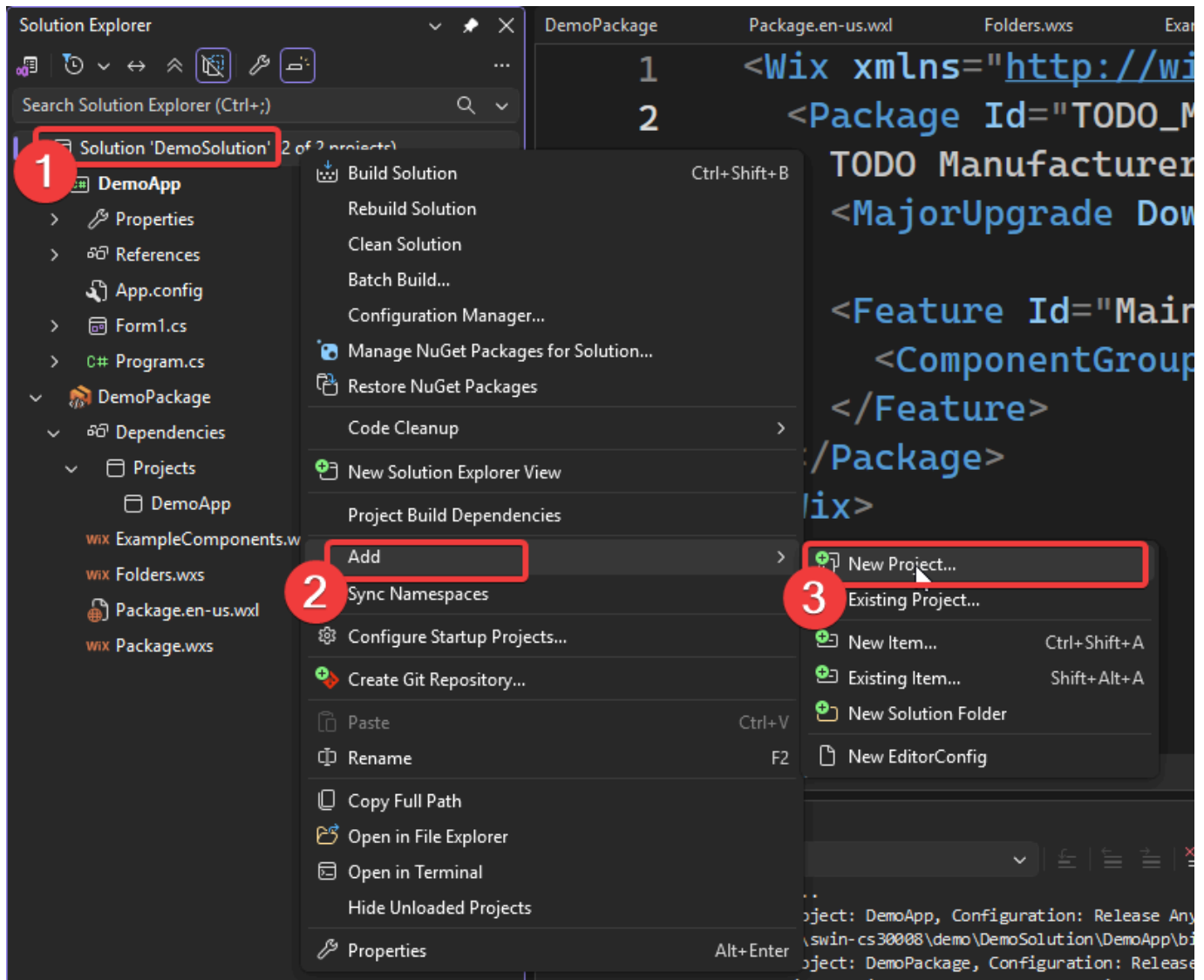
Name *

[Check availability](#)

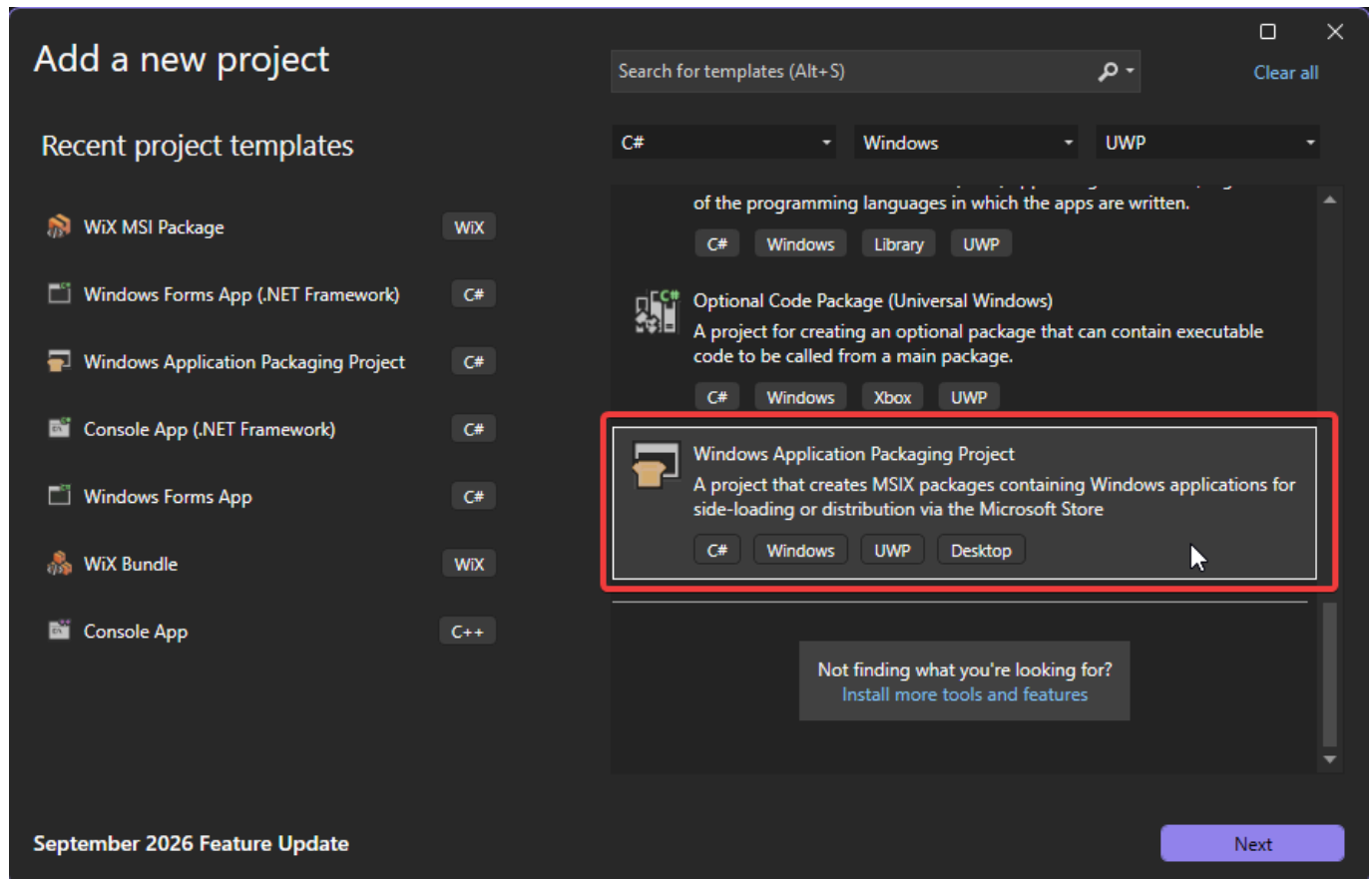
After reserving your app name, you have this screen and we will back to upload the package.



2. Back to Visual Studio, right click on the Solution in the Solution Explorer and go "Add" --> "New Project"

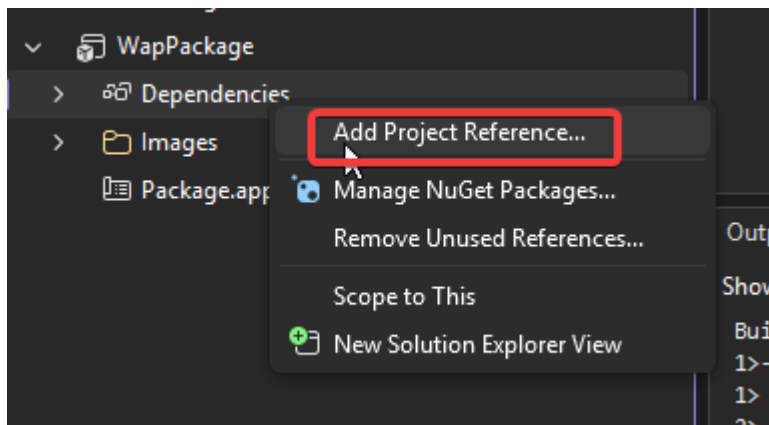


3. In the new window, click "Visual C#" and "UWP" in the sidebar. Then select "Windows Application Packaging Project", name the packaging project, and click "OK".

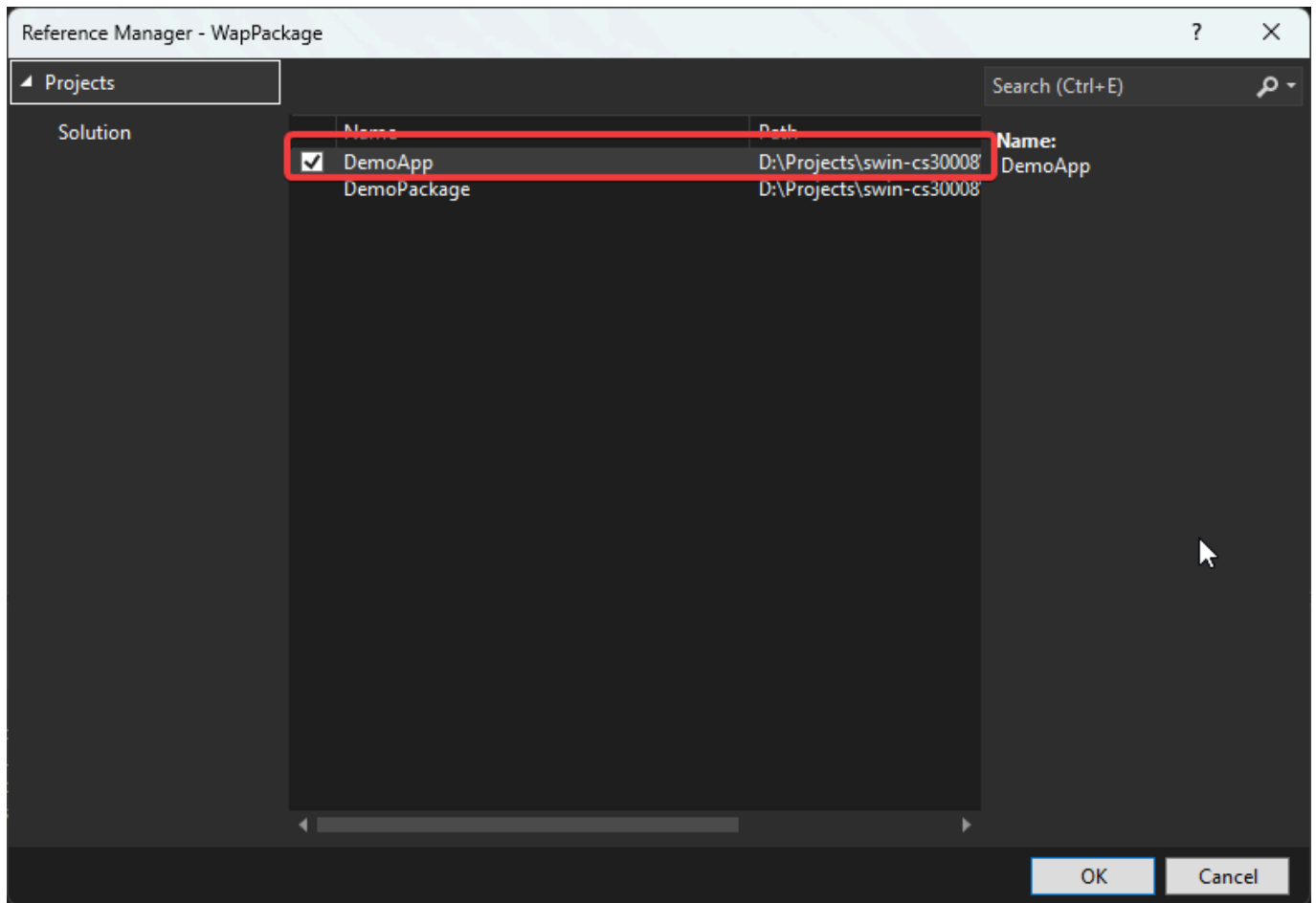


A dialog box will appear for your target version of Windows. Just accept the defaults for this project

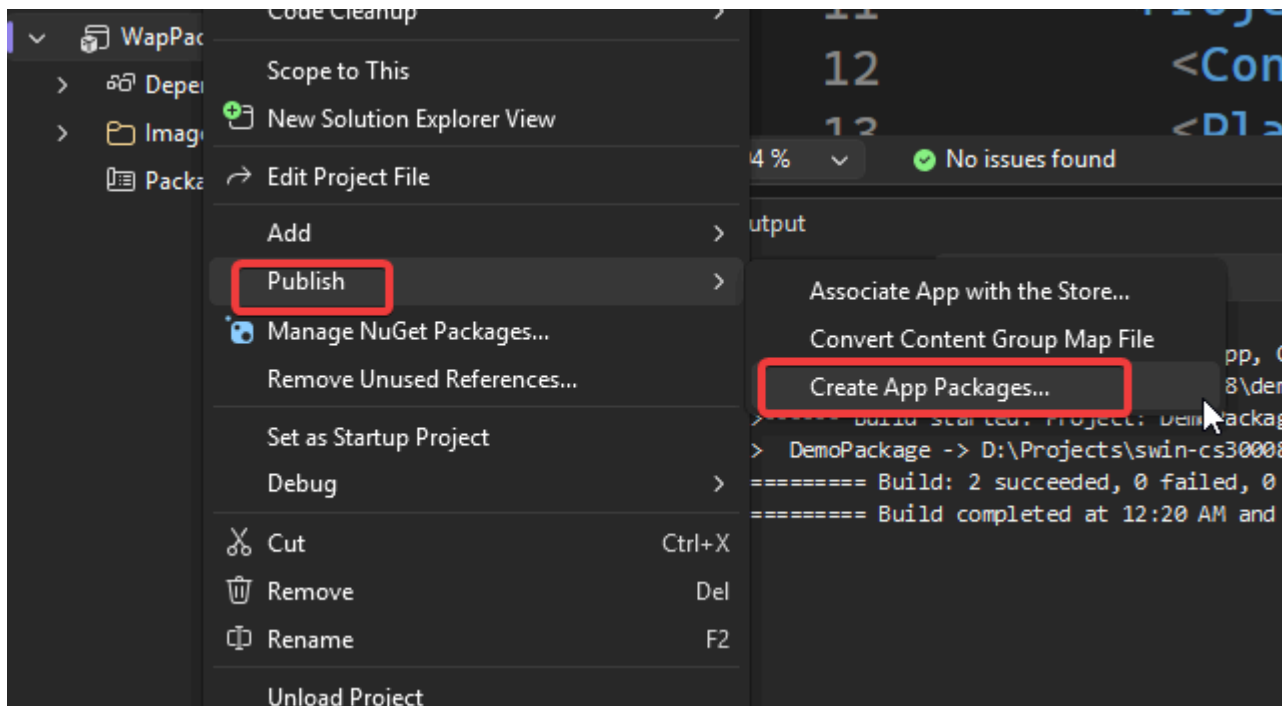
4. Add a reference to the project to build by right clicking on **Dependencies** in the solution explorer in our new Universal package project and clicking add reference



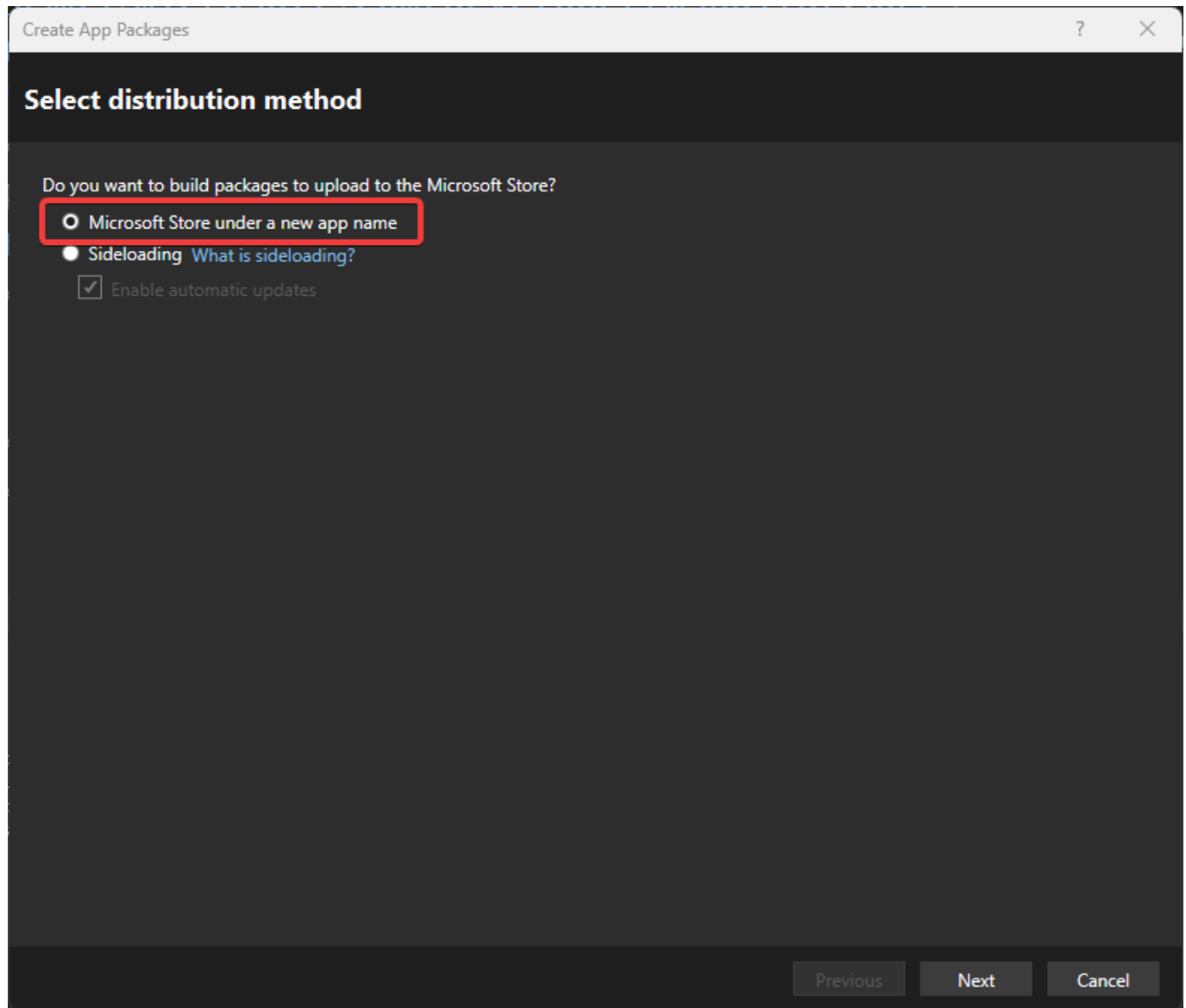
Then tick the project you want to install and click "OK"



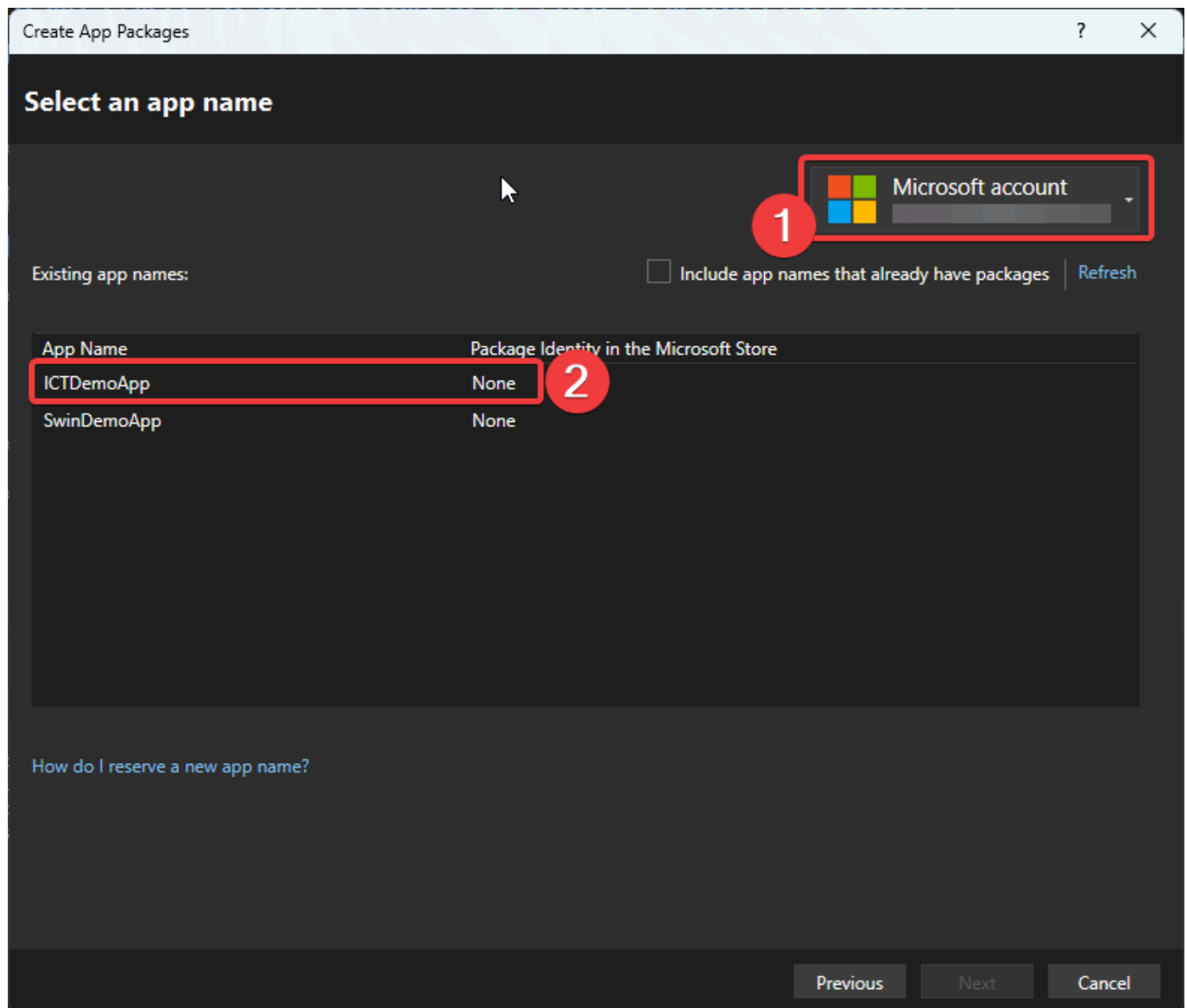
5. Right click on the project and go "Publish" --> "Create App Packages Wizard" This will open a Wizard to walk through creating the package.



6. Select Microsoft Store and click "Next" to continue with the app package creation process.



7. Login to your Developer account then Refresh, you will see your app you reserved in the submission process. Select it and click "Next" to continue.



8. Select target architectures for your app package and click "Create" to continue.

Create App Packages

Select and configure packages

Output location:
D:\Projects\swin-cs30008\demo\DemoSolution\WapPackage\AppPackages\ ...

Version:
1 . 0 . 0 . 0
☒ Automatically increment
[How do I use package version numbering?](#)

Generate app bundle:
Always ▾
[What is an app bundle?](#)

Select the packages to create and the solution configuration mappings:

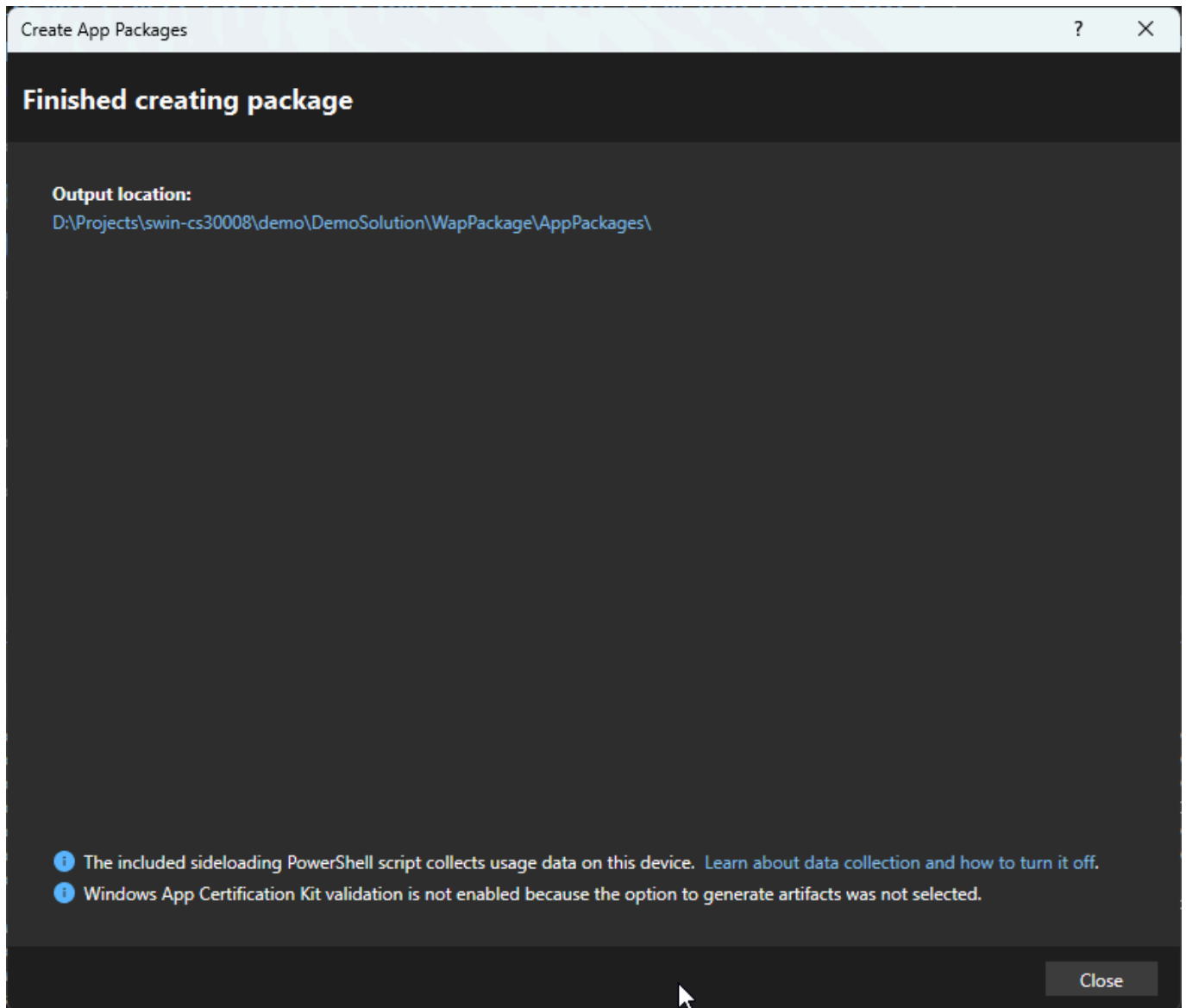
	Architecture	Solution Configuration
☐	Neutral	Release (Any CPU) ▾
☐	x86	Release (x86) ▾
☒	x64	Release (x64) ▾
☐	ARM	Release (ARM) ▾
☐	ARM64	Release (ARM64) ▾

[Select all architectures for your app to run on the most devices possible. Which devices support each architecture?](#)

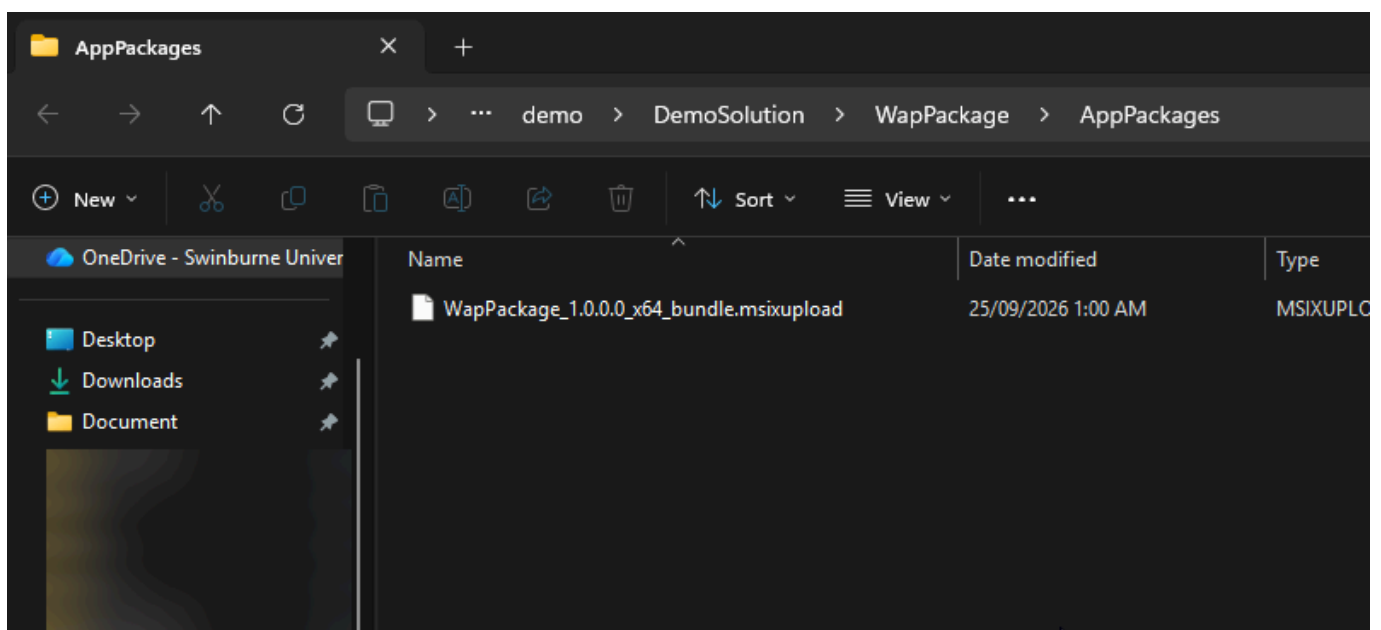
☒ Include public symbol files, if any, to enable crash analysis for the app. [How do I use debugging symbols?](#)
☐ Generate artifacts to validate the app with the Windows App Certification Kit. [More info about app package validation](#)

Previous Create Cancel

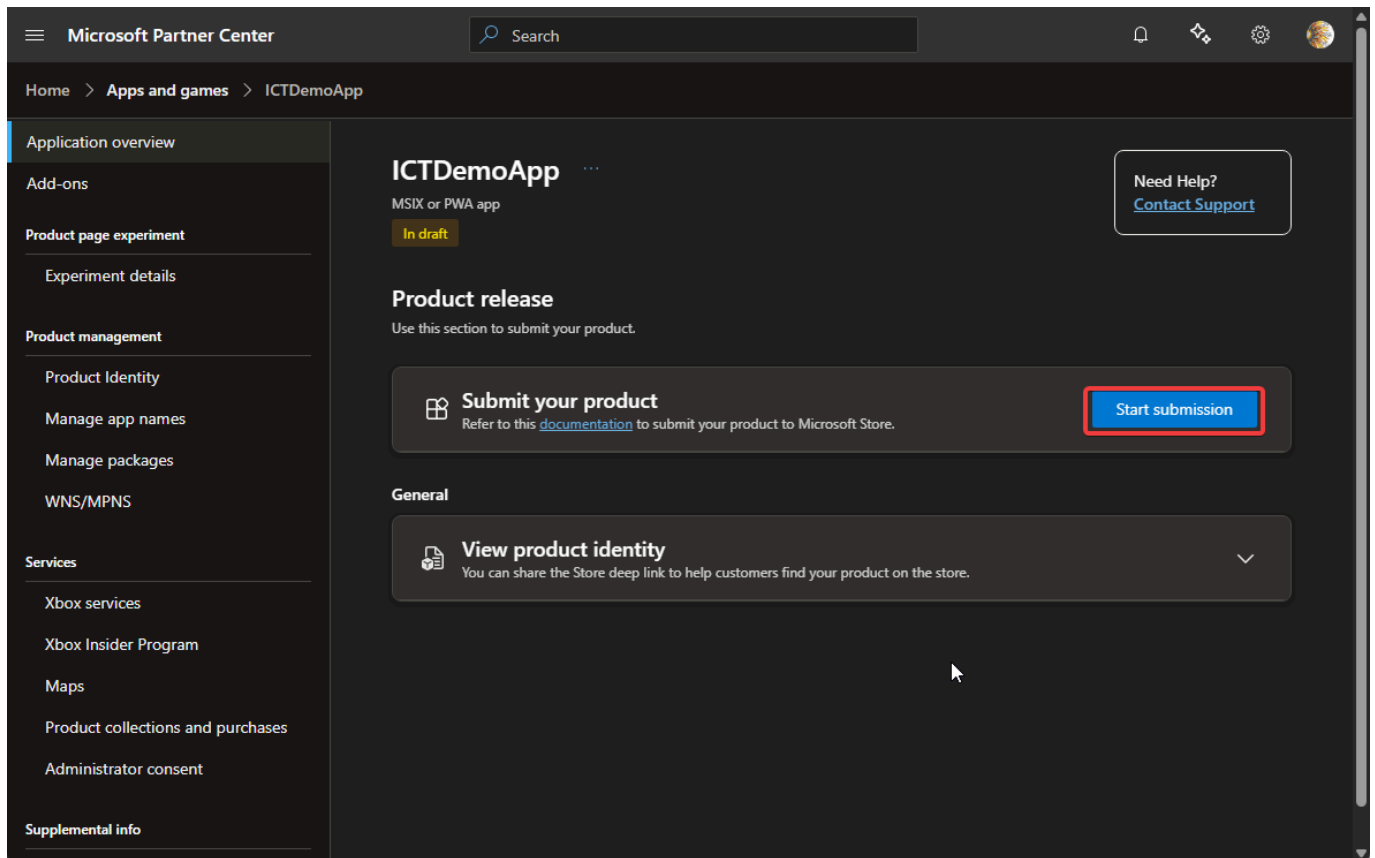
9. After clicking "Create", Visual Studio will generate the app package for you. You can then proceed to upload it to the Microsoft Store.



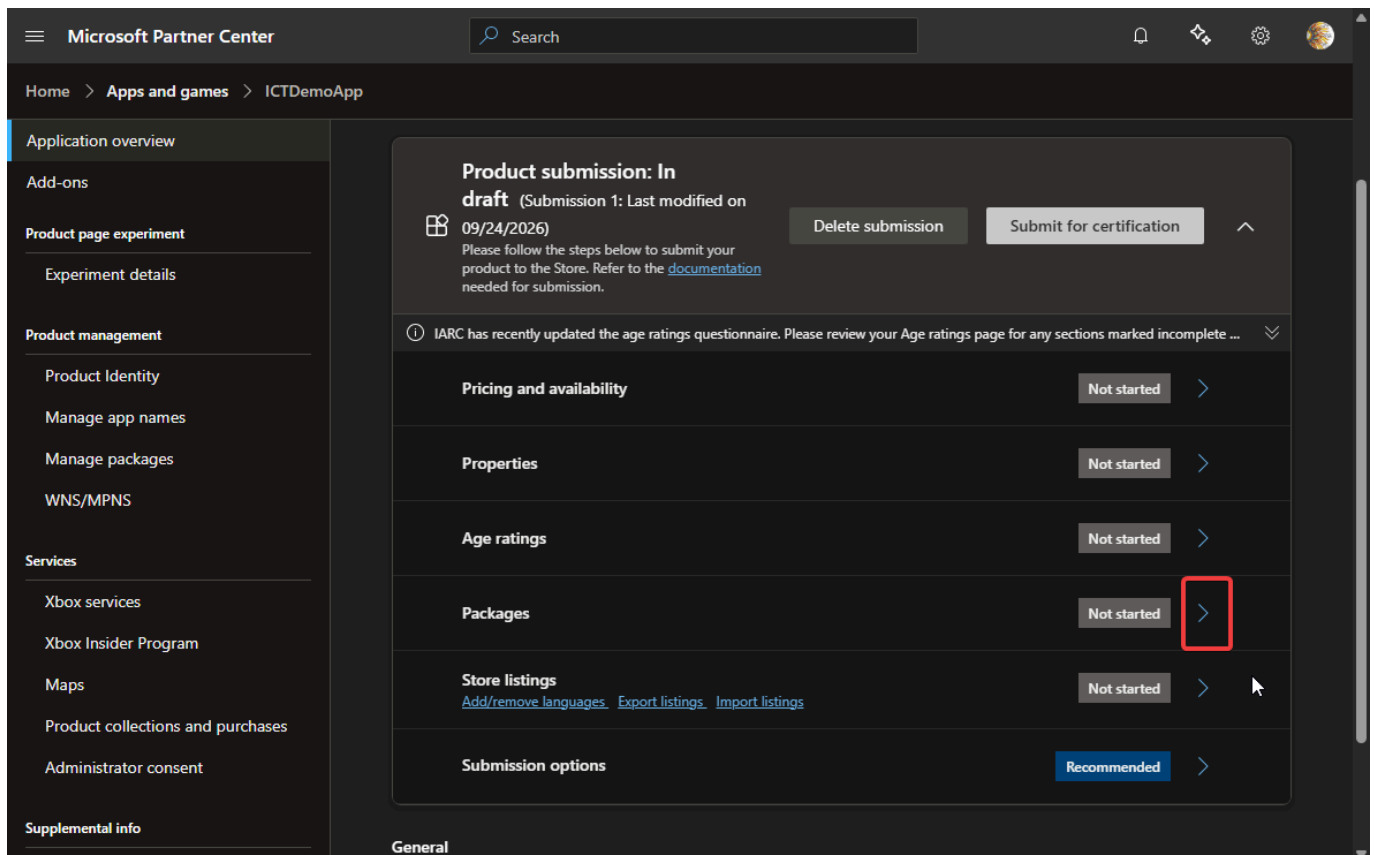
10. Go to the output folder and upload the generated app package to the Microsoft Store.



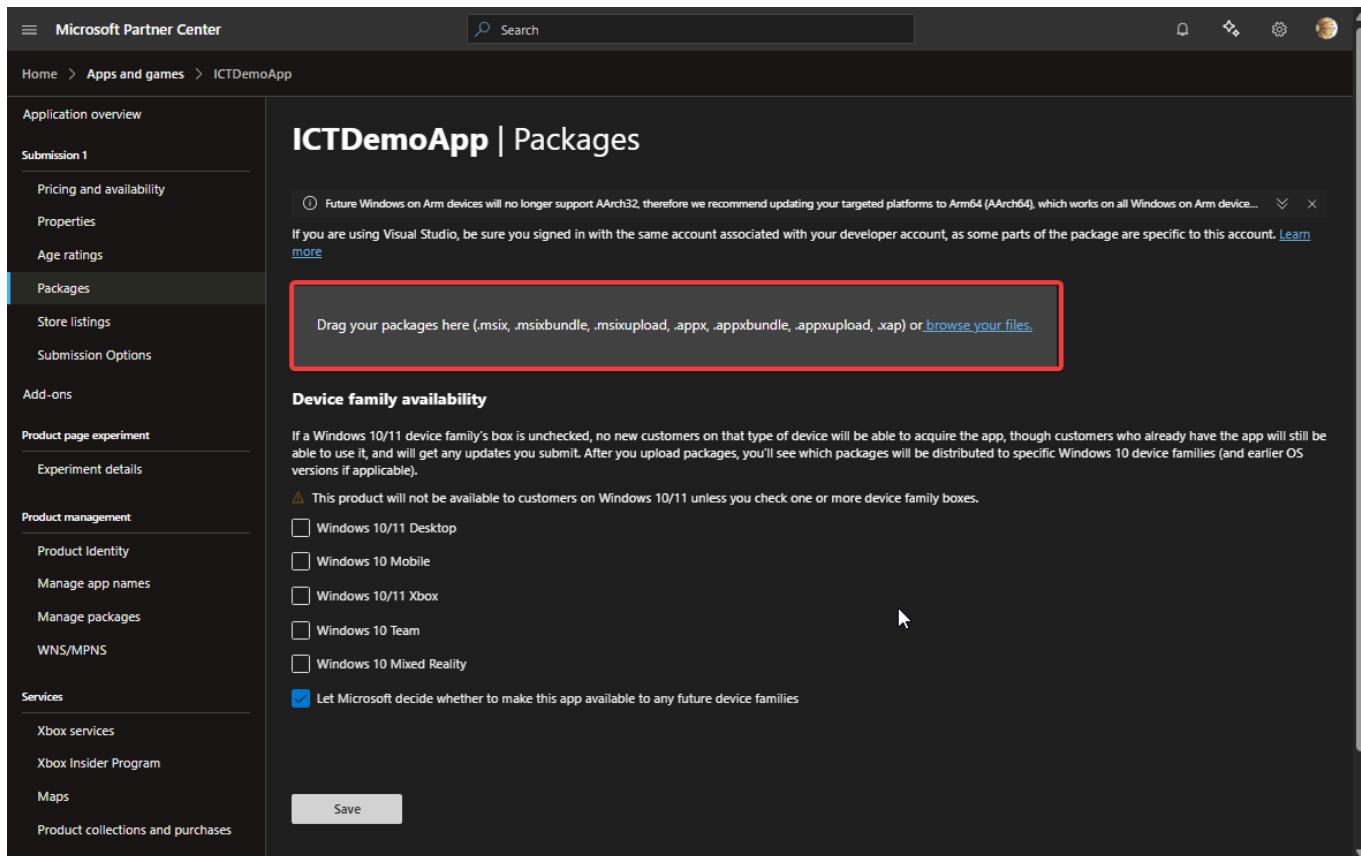
11. Back to Microsoft Partner Center, navigate to your app submission. Start submitting your app



12. Select Packages



13. Upload the app package you generated earlier and save your changes.



14. You have to complete all the required fields and steps in the app submission process before you can publish your app.

